

# Clustering of Time-Series Data Streams

Pedro Rodrigues, João Gama and João Pedro Pedroso

Artificial Intelligence and Computer Science Laboratory  
University of Porto - Portugal  
{prodrigues, jgama}@liacc.up.pt  
jpp@dcc.fc.up.pt

**Abstract.** This paper presents a time-series whole clustering system that incrementally constructs a tree-like hierarchy of clusters. The Online Divisive-Agglomerative Clustering (ODAC) system uses a correlation-based similarity measure between time-series over a data stream. When turning a leaf into a node, the cluster is divided in two and new leaves start new computations. An agglomerative phase is used to enhance a dynamic behavior capable of concept drift detection. This procedure also starts new computations, performing a cut on selected nodes. These features create the concept of fixed-period nodes which represent the state of corresponding time-series in a particular place in time. Main features include a splitting criteria supported by a significance level and an agglomerative phase based on the diameters of existing clusters. At each new example, only the leaves are updated, reducing computation of unneeded dissimilarities and speeding up the process every time the structure grows. The dissimilarity matrices are updated only for the leaves currently being tested, every  $n_{min}$  examples have been fed to the leaf. Experimental results suggest competitive performance on clustering time-series and show that the system is equivalent to a batch divisive clustering on stationary time-series, being also capable of dealing with concept drift.

## 1 Introduction

In recent days, information has grown importance as the web spread out and communications improved to a global network interaction. Facing this nowadays-world feature, efficient information extraction from data as become a major goal for all sorts of applications. One of the usual data mining problems is data clustering, either example clustering or time-series whole clustering. The aim of this work is the study of time-series whole clustering, which is meaningful according to [1]. Lack of information forced gathering data to be one of the most difficult tasks in traditional data mining applications. Nowadays, we face the opposite situation. In fact, not rarely the amount of data available from a given source is so high that traditional batch systems, based on multiple reading on same data, cannot prove themselves efficient. Moreover, in recent real-world applications the data flows continuously from a *data stream* at high speed, producing examples over time, usually one at a time. In this context, quicker responses are usually requested, for finding out pattern changes in data opens the possibility of making better decisions. The problem we address here is how to hierarchically cluster a fixed number of time-series, with new values arriving continuously over time. In

this paper we present ODAC, a time-series whole clustering system that incrementally constructs a hierarchy of clusters from a divisive point of view.

In the next section we present some related work proposed in different fields of research on clustering and data streams. Section 3 describes our proposed method, its features and objectives. Experimental results are evaluated on section 4, while section 5 presents some concluding remarks and future work.

## 2 Related Work

Among the data mining community, there's no fixed definition for the clustering procedure. Nevertheless, Guha et. al. [2] presented clustering as: *given  $n$  data points in a  $d$ -dimensional metric space, partition the data points into  $k$  clusters such that the data points within a cluster are more similar to each other than data points in different clusters*. To apply clustering to time-series (variables) rather than applying it to examples, we must consider the variables to analyze as the  $n$  data points and  $d$  equals the number of examples.

It is known that solving a clustering problem is equivalent to find the global optimal solution of a non-linear optimization problem, hence NP-hard [3], suggesting the application of optimization heuristics. Many clustering techniques emerged from research, and we can clearly group most of them into two major paradigms: *partitional* and *hierarchical* approaches [4]. Nevertheless, *density-based* [5] and *grid-based* [6–9] systems can also be found, and should be analyzed separately. *Partitional* procedures tend to minimize a cost function, while trying to achieve the optimality criterion of the model. Popular approaches in this group are the K-means [10, 11], Simulating Annealing [12], PAM [13], CLARA [13] and CLARANS [14]. *Hierarchical* models usually set their basis on a dissimilarity matrix of elements from the same cluster to construct a hierarchy of clusters (eg. AGNES and DIANA [13]).

### 2.1 Hierarchical Clustering

These approaches have three major advantages over partitional methods: a) don't require user-predefined number of target clusters; b) don't make any assumptions about data distribution; c) don't need any explicit representation rather than a pair-wise dissimilarity matrix. From these, divisive methods seem to be the most appropriated to data stream applications, as they perform a top-down strategy.

There are no restrictions on the measure used to compute the dissimilarities, besides that it should comply with distance metrics features [4]. However, calculations of the dissimilarity matrix can be proved to perform poorly when data set size enlarges or there is overlapping in clusters' definitions [15]. For hierarchical models, at each level of the hierarchy, split can be based on K-means techniques (with  $k = 2$ ) or Expectation-Maximization [16]. Another type of relevant models are Kohonen's Self-Organizing Maps [6] for which some hierarchical variants have already been proposed [17].

DIANA (DIvisive ANALysis) is a hierarchical clustering technique that constructs the hierarchy from the top (large cluster) to bottom (several clusters). Initially, there is one large cluster which contains all  $n$  variables. At each step, the cluster with the largest

diameter is selected and divided into two new clusters until all clusters contain only one variable ( $n - 1$  steps). In agglomerative clustering, for  $n$  objects, there are  $\frac{n(n-1)}{2}$  possible fusions of two different objects; in divisive clustering, there are  $2^{n-1} - 1$  possibilities of splitting the data in two clusters. To avoid considering all hypothesis, DIANA uses an heuristic approach, finding a pivot member for a splinter group and moving there the objects that are, on average, more similar to the splinter group than to the old group.

## 2.2 Incremental Clustering

Knowledge discovery systems are usually constrained by three limited resources: time, memory and sample size. In traditional applications sample size limitation proves to be dominant, since it often leads to overfitting. Nowadays, time and memory seem to be the bottleneck for machine learning applications, mainly the last one. For this reason, traditional memory-based models with multiple passes over the data are virtually impossible to achieve high performances, failing to adapt to the high-speed production of examples [18]. A system developed under the paradigm of data stream clustering must possess a compact representation, process new examples fast and incrementally, do only one (or less) pass over the entire data set, have an answer available at any time and operate inside a limited RAM buffer [19, 20].

Several methods have been already proposed in this area, covering different types of clustering. *COBWEB* [21] is a conceptual, incremental and hierarchical clustering algorithm that performs a *hill-climbing* search on classifiers. Another incremental and hierarchical approach was proposed by Zhang et. al. – the *BIRCH* system [22] – that builds a hierarchical structure of data, the *CF-tree*, by an agglomerative procedure. Bradley et al. developed the *Single Pass K-Means* [20], which uses a buffer to keep all points in a compressed form, later simplified by Farnstrom et al. [23]. At the same time, Guha et al. developed the *CURE* system where each group is represented by a constant number of points well distributed within the cluster, being capable of finding groups of arbitrary shape [2]. Barbará proposed a different approach, based on *fractal theory*, where objects are assigned to the cluster where this assignment produces less *fractal impact* [24], defining groups of objects with high self-similarity. The *STREAM* algorithm minimizes the sum-of-squares quality measure while keeping as restriction the memory available [25]. More recently, Aggarwal et al. proposed the *CluStream*, where an online component of micro-clustering extends the *CF* vectors of *BIRCH*, while an offline component uses this new information to interactively perform global clustering [26].

Unfortunately, most of the work concentrated on example clustering rather than variable clustering which, for batch processing of stationary data, is trivially transformed by doing a matrix transpose on data, but for online processing of data streams raises a whole new paradigm of clustering algorithms. One problem that usually arises with these sort of models is the definition of a minimum number of observations that are necessary to assure convergence. Techniques based on the Hoeffding bound [27] can be applied to solve this problem, and have in fact been successfully used in online decision trees [18, 28].

### 3 Online Divisive-Agglomerative Clustering

In this section we present ODAC (Online Divisive-Agglomerative Clustering), a time-series whole clustering system, based on DIANA, that incrementally constructs a binary tree-based hierarchy of clusters from a divisive point of view. Each example is processed only once. At each new example, only the leaves are updated, reducing computation of unneeded dissimilarities and speeding up the process every time the structure grows. The dissimilarity matrices are updated every  $n_{min}$  examples, but only for the leaves currently being tested for splitting or aggregation. When turning a leaf into a node, new leaves start new computations, ignoring old information. An agglomerative phase is used to enhance a dynamic behavior capable of concept drift detection [29]. This procedure also starts new computations, performing a cut on selected node. These features create the concept of fixed-period nodes which represent the state of corresponding time-series in a particular place in time. Main features include a splitting criteria supported by a significance level given by the Hoeffding Bound and an agglomerative phase based on the diameters of existing clusters.

---

#### ODAC Algorithm

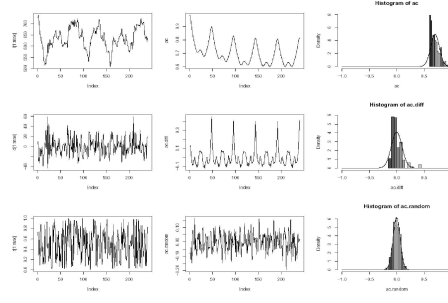
1. While(true)
  2. Read new examples and keep updating sufficient statistics on leaves.
  3. At each  $n_{min}$  examples, test each leaf for aggregation (calling TestAggregate()). If a leaf was not aggregated, test it for splitting (calling TestSplitting()).
  4. Wend
- 

#### 3.1 Dissimilarity Measure

As previously stated, only a dissimilarity matrix is needed to perform divisive analysis clustering. As we want to use the Hoeffding bound forward in the system, the evaluated function (in this case, the distance measure) must be scaled. We use Pearson's correlation between time-series as *similarity* measure. Deriving from the correlation between two time-series  $a$  and  $b$  calculated in [30], the factors used to compute the correlation can be updated incrementally, since we need only the sum of each time-series ( $A = \sum a_i$  and  $B = \sum b_i$ ), the sum of the squared time-series ( $A_2 = \sum a_i^2$  and  $B_2 = \sum b_i^2$ ) and the sum of time-series  $a \times b$  ( $A_{\times} B = \sum a_i b_i$ ), achieving an exact incremental expression for the correlation:

$$corr(a, b) = \frac{A_{\times} B - \frac{AB}{n}}{\sqrt{A_2 - \frac{A^2}{n}} \sqrt{B_2 - \frac{B^2}{n}}} \quad (1)$$

$A$ ,  $B$ ,  $A_2$ ,  $B_2$  and  $A_{\times} B$  are the *sufficient statistics* needed to perform clustering, which are easily updated at each time step. Several functions could be chosen for dissimilarity measure, if one can assure that it fulfills the three axioms that define a metric [4]. In



**Fig. 1.** Auto-correlation Study

ODAC, the dissimilarity between variables  $v_a$  and  $v_b$  is given by an appropriate metric, the *Rooted Normalized One-Minus-Correlation*

$$rnomc(v_a, v_b) = \sqrt{\frac{1 - \text{corr}(v_a, v_b)}{2}} \quad (2)$$

with range  $R = 1$ . As in DIANA, we consider the highest dissimilarity between two time-series belonging to the same cluster (or the variable variance for single variables) as the cluster's *diameter*.

The use of the system in a data stream approach is achieved by incrementally update, at each new example, the sufficient statistics needed to compute the dissimilarity matrix, which is only computed at each  $n_{min}$  examples for each leaf.

### 3.2 Splitting Criteria

DIANA always splits clusters into single objects. To prevent this behavior, the splitting criteria is enhanced and the Hoeffding bound must now be observed. For  $n$  independent observations of variable  $v_k$  with mean  $\bar{v}_k$  and range  $R$ , the Hoeffding bound states that with probability  $1 - \delta$ , the true mean of the variable is at least  $\bar{v}_k - \epsilon$ , with  $\epsilon$  given by

$$\epsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}} \quad (3)$$

The Hoeffding bound has the property that it is independent of the probability distribution generating the observations [18].

By definition, a time-series is not an independent variable, as it lives from its auto-correlation, so neither is the dissimilarity measure. Although the Hoeffding bound is usually robust enough to this kind of assumption, we decided to model first-order differences instead in order to prevent a larger error. In figure 1 we can see the graphs for a highly auto-correlated variable, its autocorrelation values and its histogram (top), and respectively the same for its first-order differences (middle) and a random variable (bottom). These results show that modeling first-order differences reduce the autocorrelation, converging to a random variable.

Many splitting tests could be applied using the confidence level given by the Hoeffding bound, but we found the forthcoming the most stable one. We introduce a parameter to the system,  $\alpha$ , which determines how sparse a cluster may be. We decide to turn a leaf into a node if  $\max(d) - \alpha \cdot \min(d) > \epsilon$ . If the Hoeffding bound isn't verified, then we can state that all variables in this cluster are almost equally distant from each other, and this is a *closed* cluster. When a split point is reported, the pivots are variables  $v_x$  and  $v_y$  where  $d(x, y) = \max(d)$ , and the usual divisive process can be performed, assigning each variable to the cluster which has the closest pivot.

---

#### TestSplit Algorithm

1. Update dissimilarities and the Hoeffding bound  $\epsilon$  for this leaf.
  2. Let  $d(x, y) = \max(d)$ . If  $d(x, y) - \alpha \cdot \min(d) > \epsilon$  then create new leaves  $l_1$  and  $l_2$ , with  $x$  and  $y$  as pivots, respectively.
  3. For each variable  $v$  in this cluster, assign it to the new leaf which has the closest pivot.
- 

An important feature of this algorithm is that every time a split is performed on a leaf with  $k$  variables, the global number of dissimilarities needed to be computed at the next iteration diminishes at least  $k$ .

### 3.3 Aggregation Criteria

For each given leaf  $C_k$ , we should search if older split decision still represents the structure of data. This way, we shall test the diameters of  $C_k$ ,  $C_k$ 's sibling and  $C_k$ 's parent, using the same  $\alpha$  parameter. Let  $C_j$  be  $C_k$ 's parent and  $C_s$  be  $C_k$ 's sibling, if the gain in diameter  $\text{diam}_k + \text{diam}_s > \text{diam}_j + \epsilon_j$ , with  $\epsilon_j$  given by the Hoeffding bound for node  $C_j$ , then we cut  $C_k$  and  $C_k$ 's sibling, decreasing the number of clusters, assuming that previous division no longer reflects the best divisive structure of data. The resulting leaf starts new computations.

---

#### TestAggregate Algorithm

1. Update dissimilarities for this leaf  $C_k$  and its sibling, if needed.
  2. Let  $C_j$  be  $C_k$ 's parent and  $C_s$  be  $C_k$ 's sibling.  
If  $\text{diam}_k + \text{diam}_s > \text{diam}_j + \epsilon_j$ , cut  $C_k$  and  $C_s$  turning  $C_j$  into a leaf with reseted sufficient statistics.
- 

## 4 Experimental Results

For a first evaluation of the proposed system, results were gathered using both synthetic and real data sets. We chose some types of data set that our system should process, creating artificial time-series with that purpose. Afterwards, we applied our system to a couple of real data sets, comparing results with a batch DIANA system, using the same correlation-based distance measure.

## 4.1 Time-Series Generator

We have created a data generator which produces  $n$  time-series belonging to a predefined number  $k$  of clusters with a noise constant  $\beta$ . Each cluster  $C_k$  has a pivot time-series  $p$ , and the remaining time-series are created as  $p + \lambda$  where  $\lambda \sim U(-\beta p, \beta p)$ . In all experiments, ODAC's  $n_{min}$  parameter was set to 1000. Output graphics show squared objects for leaves and round nodes. Information inside each node represents: node number, diameter for that cluster, number of examples seen and, for leaves only, the variables within.

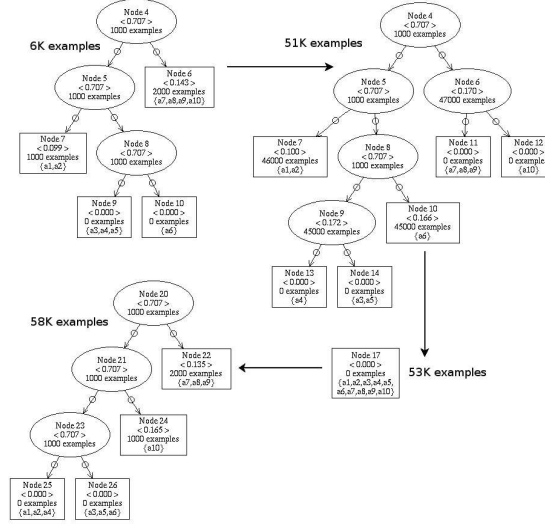
*Parameter Sensitivity* In order to find the best value for  $\alpha$ , we developed a stationary data set with four clusters and 100K examples, with  $\beta = 0.10$ , with the following structure:  $\{\{1,2\},\{3,4,5\},\{6\},\{7,8,9,10\}\}$ . As it stabilized after  $\alpha = 1.4$ , with only the absolutely necessary splits and aggregations, we decided to use this value from then on.

*Closed Set* A hierarchical clustering procedure should always create a hierarchical structure of clusters. Nevertheless, if a data set represents a closed cluster, with all variables highly correlated, then we expect a single cluster as the system output. A set of ten time-series with 100K examples was created, and we tested different values for  $\beta$ . The final result was a single cluster, although for  $\beta \geq 0.10$  some splitting might have occurred, quickly reversed by aggregation. For values of  $\beta > 0.15$  results were spoiled, as the cluster spread out (average dissimilarity  $> 0.2$ ), growing an unstable hierarchy of clusters. On forward experiences,  $\beta = 0.10$  was used.

*Sparse Set* Another plausible scenario could be a set of variables with nothing in common. For this purpose, we built a data set of ten time-series, without any relation between them, along 100K examples. Our system produced a single cluster with all variables, as it could not find a cut point, a group of variables closer to each other than the rest.

*Two Clusters* In order to study the splitting capabilities of ODAC when in presence of a simple cut point, we built a data set with ten time-series divided in two clusters. The correct outcome was achieved immediately.

*Concept Drift Detection* For a more complex evaluation of system's dynamics (splitting and aggregation), we created a data set where the first 50K examples had the structure used on parameter sensitivity tests, and the second half had the following one:  $\{\{1,2,4\},\{3,5,6\},\{7,8,9\},\{10\}\}$ . We watched the evolution of the structure over the iterations. The system found the first structure in 6 iterations and then stabilized until a change after 50K examples triggered the dynamics of the system. The first attempt the system executed was splitting two of the leaves (51K examples), as the variables started to diverge. After that, the aggregation took place and the final structure was found after only 8 iterations since real concept drift. Selected transitory structures are sketched on figure 2. The system became robust enough to handle higher levels of noise. Simultaneously, we observed a dynamic behavior of the system, splitting and aggregating along the examples, until it stabilizes on the final tree structure. These results suggest good performance on splitting, concept drift detection and aggregating.



**Fig. 2.** Evolution on Concept Drift

## 4.2 Real Data

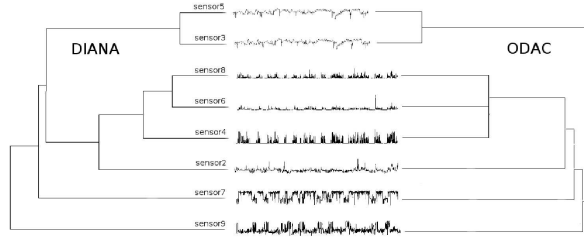
For a good performance on experimental results, every system should be tested using real world data. This section introduces two data sets from different applications on real world data mining.

**PDMC** The Physiological Data Modeling Contest [31] was held at ICML 2004, as a workshop, and aimed at information extraction from streaming sensors data. The training data set for the contest<sup>1</sup> consisted of approximately 10,000 hours of this data, containing several variables: userID, sessionID, sessionTime, characteristic[1..2], annotation, gender and sensor[1..9]. We concentrated on sensors 2 to 9, and extracted all data by userID, ending with several data sets of eight continuous variables. Finally, we tried to find some cluster structure on these variables, and compare it with a batch divisive analysis system, using the same dissimilarity measure for each user. For  $userID = 1$ , 93344 examples were processed with  $n_{min} = 1000$ . Our system evolved splitting and aggregating, until a stable structure was gathered after 55K examples, never changing from then on. Comparison with batch system is shown in figure 3. For other userIDs, results were very similar.

**Electrical Demand Data Set** Time-series of electrical data are one of the most widely studied sets of data, mainly for forecasting purposes, usually using neural networks. One big problem with this approach is the time consumption of neural network's training procedure. This fact, in conjunction with a high dimensionality (common electrical

<sup>1</sup> available at <http://www.bodymedia.com/support/TrainingSet.zip>





**Fig. 3.** Comparison for user01 data set - final result is approximately equivalent to the structure achieved by the batch system using the same dissimilarity measure.

networks combine thousands of different time-series) suggests the need to cluster time-series. Usually, an expert is required for this job. Our system may present some benefits in this particular task. From the raw data received at each sub-station, we have extracted only the values related to load intensity, five minutes each example, which ended on a data set with 923 variables and 907 examples. Setting the parameters at  $\alpha = 70$  (this value decreases the number of clusters as it keeps them wide) and  $n_{min} = 10$ , the final structure can be analyzed on figure 4. The system found a couple of big clusters, and splintered some variables away, as a human expert would probably do. Nevertheless, the real structure is too complex to be achieved with this value. Using the value gathered with parameter sensitivity test ( $\alpha = 1.4$ , a huge structure was found, with 800 splits and 461 aggregations, resulting on an almost balanced tree, from which small clusters can be obtained. One of the clusters is shown on figure 5.

## 5 Conclusions and Future Work

This work presents a time-series whole clustering system that works online. The ODAC system constructs a hierarchy of clusters incrementally from a divisive point of view, and is capable of adaptation to changes in the time-series by means of posterior agglomeration. In ODAC, examples are processed as they arrive, using a single scan over the data. The sufficient statistics needed to incrementally compute the dissimilarities are maintained and updated at each new example, updating only the leaves, thus reducing computation of unneeded dissimilarities, speeding up the process along structure growth. When created, new leaves start new computations, forgetting old information for concept drift detection purposes. These features create the concept of *fixed-period* nodes which represent the state of time-series in a particular place in time. One important feature of the system is that the dissimilarity matrix is updated only for the cluster currently being tested, diminishing the global number of dissimilarities needed to be computed at each step. The system uses a correlation-based distance as *dissimilarity* measure and supports the splitting decision on a significance level given by the Hoeffding bound. Other main features include a splitting criteria supported by a significance level and an agglomerative phase based on the diameters of existing clusters. Experimental results suggest competitive performance on clustering time-series



and show that the system is equivalent to a batch divisive clustering on stationary time-series, being also capable of dealing with concept drift. The experimental results suggest that the system exhibit dynamic behavior, adapting to changes in time-series. Future work will focus on a complete system for high number of load forecast time-series, in the presence of charge transfers [32], and concept drift [29, 28] to improve the search for changes in cluster structure. The agglomeration phase must be better studied, in order to prevent cyclic behaviors.

*Acknowledgments* The authors reveal their gratitude to the financial support given by the FEDER, and the *Plurianual* support attributed to LIACC. This work was developed in the context of projects RETINAE (IDEIA/ADI 70/00078) and ALES II (POSI/EIA/55340/2004).

## References

1. Keogh, E.J., Lin, J., Truppel, W.: Clustering of time series subsequences is meaningless: Implications for previous and future research. In: Proceedings of the IEEE International Conference on Data Mining, IEEE Computer Society Press (2003) 115–122
2. Guha, S., Rastogi, R., Shim, K.: CURE: an efficient clustering algorithm for large databases. In Haas, L.M., Tiwary, A., eds.: SIGMOD 1998, Proceedings ACM SIGMOD International Conference on Management of Data, ACM Press (1998) 73–84
3. Bern, M., Eppstein, D.: 8. Approximation algorithms for geometric problems. In: Approximation Algorithms for NP-hard Problems. WS Publishing (1996) 296–345
4. Han, J., Kamber, M.: Data Mining: Concepts and Techniques. Morgan Kaufmann (2001)
5. Ester, M., Kriegel, H.P., Sander, J., Xu, X.: A density-based algorithm for discovering clusters in large spatial databases with noise. In Simoudis, E., Han, J., Fayyad, U., eds.: Second International Conference on Knowledge Discovery and Data Mining, Portland, Oregon, AAAI Press (1996) 226–231
6. Kohonen, T.: Self-Organizing Maps. 3rd edn. Volume 30 of Springer Series in Information Sciences. Springer, Berlin (2001)
7. Wang, W., Yang, J., Muntz, R.R.: STING: A statistical information grid approach to spatial data mining. In Jarke, M., Carey, M.J., Dittrich, K.R., Lochovsky, F.H., Loucopoulos, P., Jeusfeld, M.A., eds.: Twenty-Third International Conference on Very Large Data Bases, Athens, Greece, Morgan Kaufmann (1997) 186–195
8. Sheikholeslami, G., Chatterjee, S., Zhang, A.: WaveCluster: A multi-resolution clustering approach for very large spatial databases. In: Proceedings of the 24th International Conference on Very Large Data Bases, VLDB, New York, USA (1998) 428–439
9. Agrawal, R., Gehrke, J., Gunopulos, D., Raghavan, P.: Automatic subspace clustering of high dimensional data for data mining applications. In: Proceedings of the ACM-SIGMOD International Conference on Management of Data, Seattle, Washington (1998) 94–105
10. MacQueen, J.B.: Some methods for classification and analysis of multivariate observations. In LeCam, L.M., Neyman, N., eds.: Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Berkeley, University of California Press (1967) 281–297
11. Bradley, P.S., Mangasarian, O.L., Street, W.N.: Clustering via concave minimization. In Mozer, M.C., Jordan, M.I., Petsche, T., eds.: Advances in Neural Information Processing Systems. Volume 9., The MIT Press (1997) 368
12. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. *Science* **220** (1983) 671–680

13. Kaufman, L., Rousseeuw, P.J.: Finding Groups in Data: An Introduction to Cluster Analysis. John Wiley and Sons, Inc., New York (1990)
14. Ng, R.T., Han, J.: Efficient and effective clustering methods for spatial data mining. In Bocca, J., Jarke, M., Zaniolo, C., eds.: Proceedings of the 20th International Conference on Very Large Data Bases, Morgan Kaufmann Publishers (1994) 144–155
15. Fasulo, D.: An analysis of recent work on clustering algorithms. Technical Report UW-CSE01-03-02, University of Washington (1999)
16. Heckerman, D., Meek, C., Thiesson, B.: Accelerating em for large databases. Technical report, Microsoft Corp. (1999)
17. Pampalk, E., Widmer, G., Chan, A.: A new approach to hierarchical clustering and structuring of data with self-organizing maps. *Intelligent Data Analysis* (2004) 131–149
18. Domingos, P., Hulten, G.: Mining high-speed data streams. In: Proceedings of the Sixth International Conference on Knowledge Discovery and Data Mining, Boston, MA, ACM Press (2000) 71–80
19. Barbará, D.: Requirements for clustering data streams. *SIGKDD Explorations (Special Issue on Online, Interactive, and Anytime Data Mining)* **3** (2002) 23–27
20. Bradley, P., Fayyad, U., Reina, C.: Scaling clustering algorithms to large databases. In: Proceedings of the Fourth International Conference on Knowledge Discovery and Data Mining, AAAI Press (1998) 9–15
21. Fisher, D.H.: Knowledge acquisition via incremental conceptual clustering. *Machine Learning* **2** (1987) 139–172
22. Zhang, T., Ramakrishnan, R., Livny, M.: BIRCH: an efficient data clustering method for very large databases. In: Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data. (1996) 103–114
23. Farnstrom, F., Lewis, J., Elkan, C.: Scalability for clustering algorithms revisited. *SIGKDD Explorations* **2** (2000) 51–57
24. Barbará, D., Chen, P.: Using the fractal dimension to cluster datasets. In: Proc. of the 6th International Conference on Knowledge Discovery and Data Mining (KDD-2000). (2000) 260–264
25. O’Callaghan, L., Mishra, N., Meyerson, A., Guha, S., Motwani, R.: Streaming-data algorithms for high-quality clustering. In: ICDE. (2002)
26. Aggarwal, C., Han, J., Wang, J., Yu, P.: A framework for clustering evolving data streams. In: Proceedings of the 29th VLDB Conference. (2003)
27. Hoeffding, W.: Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association* (1963) 13–30
28. Gama, J., Medas, P., Rodrigues, P.: Learning decision trees from dynamic data streams. In Haddad, H., Liebrock, L.M., Omicini, A., Wainwright, R.L., eds.: SAC ’05: Proceedings of the 2005 ACM Symposium on Applied Computing, Santa Fe, New Mexico, USA, ACM Press (2005) 573–577
29. Gama, J., Medas, P., Castillo, G., Rodrigues, P.: Learning with drift detection. In Bazzan, A.L.C., Labidi, S., eds.: Advances in Artificial Intelligence - SBIA 2004. Volume 3171 of Lecture Notes in Computer Science., São Luiz, Maranhão, Brazil, Springer Verlag (2004) 286–295
30. Wang, M., Wang, X.S.: Efficient evaluation of composite correlations for streaming time series. In: Proceedings of the 4th International Conference on Web-Age Information Management (WAIM 2003), Chengdu, China (2003)
31. Andre, D., Stone, P.: Physiological data modeling contest. A ICML’04 Workshop (2004) <http://www.cs.utexas.edu/users/sherstov/pdmc/>.
32. Rodrigues, P., Gama, J.: Forecast adaptation to charge transfers. *WSEAS Transactions on Circuits and Systems* **3** (2004) 1693–1699